LÓGICA DE PROGRAMAÇÃO

Resumo Expandido: Linguagens, Tradutores e Estrutura de Dados

---

Atenção

Objetivos desta Unidade de Estudo:

- Compreender a taxonomia das linguagens e o funcionamento dos tradutores.
- Analisar o panorama das linguagens de mercado e suas aplicações.
- Consolidar o uso de tipos primitivos, variáveis e constantes.
- Dominar a construção de expressões e o uso de operadores diversos.

grafo.jpeg

1. Tipos de Linguagem de Programação

As linguagens de programação são ferramentas que permitem ao ser humano instruir o hardware. Elas são classificadas principalmente pelo seu nível de abstração em relação à máquina.

1.1 Linguagens de Baixo Nível

São aquelas cujas instruções são interpretadas diretamente pelo hardware ou estão muito próximas dele.

- **Código de Máquina:** A única linguagem que o processador entende de fato, composta por sequências de bits (0 e 1).
- **Assembly:** Utiliza mnemônicos para representar operações de baixo nível. É altamente dependente da arquitetura do processador.

1.2 Linguagens de Alto Nível

Possuem uma sintaxe mais próxima da linguagem humana (geralmente o inglês) e de conceitos matemáticos. Isso permite que o desenvolvedor foque na lógica e não nos detalhes físicos do hardware.

- Exemplos clássicos: C, Pascal, Java, Python, C++, C#.

grafo.jpeg

2. Tradutores: Compiladores, Interpretadores e Assemblers

Como o computador só entende binário, qualquer código escrito em alto nível precisa ser convertido. Esse processo é realizado por softwares específicos.

2.1 Compiladores

O compilador lê todo o programa fonte e gera um código de máquina equivalente em um arquivo separado (o executável).

- **Processo:** Análise Léxica → Análise Sintática → Geração de Código Objeto.
- **Vantagem:** O programa final é mais rápido, pois a tradução é feita uma única vez antes da execução.

2.2 Interpretadores

Ao contrário do compilador, o interpretador traduz e executa o código linha por linha em tempo real.

- **Processo:** Lê a linha → Traduz → Executa → Passa para a próxima.
- **Vantagem:** Facilita a depuração de erros durante o desenvolvimento.

grafo.jpeg

2.3 Assemblers (Montadores)

São tradutores específicos para linguagens de baixo nível. Eles convertem o código em Assembly para linguagem de máquina pura.

3. Linguagens de Mercado e Suas Aplicações

O mercado atual é diversificado, com linguagens otimizadas para diferentes domínios de aplicação.

1. **Linguagem C:** A base de quase todos os sistemas operacionais. Essencial para o ensino de lógica por exigir controle explícito de memória.

2. **C++:** Uma evolução do C que adiciona orientação a objetos. Líder no mercado de jogos AAA e softwares de alto desempenho.

3. **Java:** Famosa pela portabilidade através da JVM (Java Virtual Machine). Amplamente usada em sistemas bancários e no desenvolvimento Android.

4. **C# (C-Sharp):** Principal linguagem do ecossistema Microsoft. Muito forte em sistemas corporativos e no motor de jogos Unity.

5. **Pascal:** Embora menos comum comercialmente hoje, ainda é uma referência acadêmica por sua estrutura rígida que favorece o aprendizado de algoritmos.

grafo.jpeg

6. **Python:** Atualmente a linguagem mais popular para Ciência de Dados, Inteligência Artificial e automação de tarefas.

7. **JavaScript:** A linguagem soberana do desenvolvimento Web (front-end), permitindo interatividade nos navegadores.

8. **TypeScript:** Uma extensão do JavaScript que adiciona tipagem forte, ganhando muito espaço em grandes projetos web.

9. **PHP:** Linguagem veterana mas ainda vital para o back-end da internet, alimentando plataformas como o WordPress.

4. Tipos Primitivos de Dados

Para que o computador processe as informações corretamente, ele precisa saber a natureza dos dados que está manipulando.

4.1 Categorias Fundamentais

- **Inteiro (int):** Números positivos, negativos ou zero, sem parte decimal. Usado para contagens e índices.

- **Real (float/double):** Números com ponto flutuante (decimais). Usado para medidas e cálculos financeiros.

- **Lógico (boolean):** Aceita apenas dois estados: verdadeiro ou falso. É o coração das estruturas de controle.

- **Caractere (char/string):** Letras, símbolos e cadeias de texto. Note que "123"como string não pode ser usado em somas aritméticas.

grafo.jpeg

Contexto Teórico

Definir o tipo de dado adequado é vital para economizar memória e evitar erros de processamento (como tentar somar um nome a um número).

5. Variáveis e Constantes

5.1 Variáveis

Representam espaços na memória cujo conteúdo pode ser alterado durante a execução.

> **Atenção**
> **Regras de Nomenclatura:**
>
> - Devem ser nomes mnemônicos (ex: `salario_base` é melhor que `s`).
> - Não podem conter espaços ou caracteres especiais.
> - Não podem iniciar com números.

5.2 Constantes

São identificadores cujo valor é fixado no momento da declaração e nunca se altera.

> **Exemplo/Código**
> Exemplo clássico: `const PI <- 3.14159`. Usar constantes torna o código mais legível e fácil de manter.

grafo.jpeg

6. Operadores e Expressões Aritméticas

Expressões aritméticas são combinações de variáveis, constantes e operadores que resultam em um valor numérico.

6.1 Principais Operadores

- + (Adição) e - (Subtração).
- * (Multiplicação) e / (Divisão).
- **Mod** (Resto da divisão inteira): Útil para identificar números pares ou múltiplos.
- ** ou ^ (Exponenciação).

> **Contexto Teórico**
> **Precedência Matemática:** Multiplicação e divisão são resolvidas antes de adição e subtração, a menos que existam parênteses.

grafo.jpeg

7. Expressões Relacionais e Lógicas

7.1 Operadores Relacionais

Servem para comparar dois valores, resultando sempre em um dado lógico (verdadeiro/falso).

- >, <, >=, <=.
- = (Igualdade em pseudocódigo) ou == (em C).
- <> (Diferença em pseudocódigo) ou != (em C).

7.2 Operadores Lógicos

Permitem combinar múltiplas condições em uma única expressão complexa.

- **E (AND):** Verdadeiro apenas se todas as partes forem verdadeiras.
- **OU (OR):** Verdadeiro se pelo menos uma parte for verdadeira.
- **NÃO (NOT):** Inverte o valor lógico atual.

grafo.jpeg

8. Tabela Verdade e Avaliação

A tabela verdade é a ferramenta que facilita o entendimento do uso dos operadores lógicos combinados.

A	B	A E B	A OU B	NÃO A
V	V	V	V	F
V	F	F	V	F
F	V	F	V	V
F	F	F	F	V

Exemplo/Código

Avaliação da expressão: `(10 > 5) E (8 < 3)`

1. `10 > 5` é Verdadeiro.
2. `8 < 3` é Falso.
3. `Verdadeiro E Falso` resulta em **Falso**.

grafo.jpeg

9. Exercícios Práticos de Revisão

Exercício 1: Explique por que um interpretador costuma ser mais lento na execução do que um programa compilado.

Exercício 2: Identifique qual o tipo primitivo ideal para armazenar a placa de um veículo e a temperatura de um paciente.

Exercício 3: Qual a principal diferença entre uma variável e uma constante em termos de ciclo de vida no programa?

Exercício 4: Escreva uma expressão lógica que verifique se a variável `idade` está entre 18 e 65 anos (inclusive).

Exercício 5: Determine o resultado de: `(NÃO (5 = 5)) OU (15 / 3 = 5)`.

grafo.jpeg

10. Gabarito Detalhado

Exemplo/Código

1. O interpretador precisa traduzir cada linha toda vez que ela é executada. O compilador traduz tudo uma vez só, gerando um código que a máquina entende diretamente.

2. Placa de veículo: **Literal/Caractere** (pois contém letras e números). Temperatura: **Real** (float/double), pois exige casas decimais.

3. A variável pode ter seu valor alterado várias vezes; a constante nasce com um valor e morre com o mesmo valor, garantindo imutabilidade.

4. Expressão: `(idade >= 18) E (idade <= 65)`.

5.

- `(5 = 5)` é V, então `NÃO V` é **F**.
- `(15 / 3 = 5)` é **V**.
- `F OU V` resulta em **Verdadeiro**.

grafo.jpeg